

使用哈希函数构造密码学证明

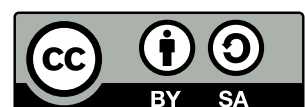
Alessandro Chiesa

Eylon Yogev

译者: Kurt Pan

June 24, 2025

This work is licensed under a Creative Commons “Attribution-ShareAlike 4.0 International” license.



Contents

前言	iv
序言	vi
1 数学背景	1

前言

简洁证明系统是一种非凡的工具。我们用一个简单的例子来简要解释它们是什么。假设爱丽丝有一个做了一些有趣事情的公共程序 P ，比如会依次输出 π 的十进制数字，首先是 3，然后是 1，然后是 4，依此类推。爱丽丝在她的笔记本电脑上运行这个程序，程序输出了 π 的第一百万位小数，这正好是 5。她非常兴奋，于是告诉了她所有的朋友。爱丽丝的一个朋友鲍勃有点怀疑。他检查了程序 P 的代码，确认它确实是在正确地计算 π 的数字。但他担心爱丽丝只是猜测了第一百万位数字是什么而并没有实际运行程序。鲍勃决定自己验证一下这个计算。他设置好他的笔记本电脑运行 P ，并等待它重新确认爱丽丝的发现。

当鲍勃等待他的笔记本电脑从头开始重新运行整个计算时，他开始觉得这毫无意义。如果他们的朋友卡罗尔也不信任他们中的任何一个，她也将不得不重新运行整个计算以说服自己 P 确实正确运行了。如果大卫不信任他们三个人中的任何一个，他也将不得不重新运行整个计算。这是大量的重复劳动。他们所有人都在一遍又一遍地重做相同的计算。

有没有更好的方法？简洁证明系统提供了一个非凡的解决方案。它使得爱丽丝能够计算出程序 P 正确运行并且其输出如所声称的那样的证明。这个证明是简洁的，意味着它很短且验证速度很快。在实践中，简洁证明只有几千字节长，验证只需几毫秒。这里应当停下来对此感到惊叹：无论爱丽丝计算的是 π 的第一百万位还是第一十亿位数字，证明她计算是正确的证明总是只有几千字节长且只需几毫秒就能验证。

令人惊奇的是，这适用于任意程序 P ，即使它需要爱丽丝提供辅助输入。爱丽丝可以运行程序并发布其输出以及程序正确运行的简洁证明。然后，只需几毫秒的工作，鲍勃、卡罗尔、大卫以及其他任何人都可以检查这个证明，并确信爱丽丝正确地运行了这个程序。他们无需重新运行计算。这个简洁证明甚至可以是零知识的，意思是证明不会透露爱丽丝的辅助输入的任何信息，但它仍然可以让所有人相信程序确实在这个输入上正确运行了。

近年来，简洁证明系统的研究已经发展成为一个庞大的研究领域，许多研究者都贡献了许多美妙的想法。推动这一快速进展的原因之一是这些工具的商业化应用，例如对去中心化系统扩容。另一个原因是这一领域为新的技术创新提供了肥沃的土壤。有多种构造简洁证明的方法，给研究者提供了许多探索的方向。同时，工程师们也有强烈的需求希望构造编程框架，使得非专家也能轻松使用这些工具。总体而言，这是一个由研究者和开发者组成的充满活力的社区，这一领域的工作既令人兴奋又充满乐趣。

本书涵盖了一种特定的简洁证明系统构造方法，称为基于哈希的证明。这些证明系统在概念上是最简单的，不需要复杂的数学知识，因此对广泛的读者群体都具有吸引力。此外，这些证明系统的安全性依赖于哈希函数的相对简单的性质。对于标准的密码学哈希函数，这些性质被认为即使在敌手拥有大规模量子计算机的情况下也能成立。因此，这些证明系统是后量子的：即使在构建出大规模量子计算机之后，它们仍将保持安全。

基于哈希的证明系统的理论已经相当成熟且被透彻理解。然而，直到现在，还没有一本书提供所有的详细信息。这就是本书的目的。本书开始部分精确定义了构成简洁证明系统的元素。接下来，本书转向从几个抽象的信息论对象构建简洁证明系统。一个广泛使用的例子是从一个重要对象——交互式预言机证明 (Interactive Oracle Proof, 简称 IOP) 构造简洁证明系统。该构造完全依赖于哈希函数。书中逐步构造必要的工具，然后清晰描述了该构造及其安全性证明。

本书是对证明系统领域的重大贡献。它适合喜欢对材料进行清晰且精确处理的读者。我相信，一旦你读了这本书，你会想要了解更多关于这个领域的知识。本书是进入简洁证明世界的一个很好的起点，带你踏上一段有趣的旅程。

Dan Boneh
2024 年 4 月
斯坦福大学

序言

“证明对我们的生活至关重要，而对于所有基本的事物，我们应该有所预料，对证明是什么的回答将永远是一个发展中的过程。”

Silvio Micali, 计算可靠证明

本书讨论的是**密码学证明**，即用于证明和验证计算的正确执行的简洁且零知识的（稍后将详细介绍）协议。密码学证明是计算机科学中的一个基本对象，处于密码学和计算复杂性理论的交汇之处。由于实际应用的推动，在学术界和工业界受到了极大的关注。密码学证明结合了计算理论中一些最优美的思想和最强大的工具，是深奥的数学思想在新技术中起关键作用的一个显著例子。

本书提供了一个基于（理想）哈希函数构造的密码学证明的严格介绍，既是学生自学的教材，也是从业者的参考书。这包括了基于（理想）哈希函数的简洁非交互论证（SNARGs）的著名构造。例如，STARKs（可扩展透明知识论证）就是此类 SNARGs 的一个例子。

最新版本 本书的最新版本可以在以下网站找到：

<https://hash-based-snargs-book.github.io/>

该网站还链接了本书源代码的 Git 库，包括最新的更正和补充内容。我们非常欢迎对本书的评论（无论正面或负面的！），以及任何的更正或建议。您可以直接在 GitHub 上的库里提交 issues 或 PR，或者直接通过电子邮件联系我们。

许可证 本书的源代码（以及本书本身）是根据 Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA 4.0) 许可的。简而言之，您可以分享和改编本书的源代码，前提是您给予适当的署名并注明任何更改；此外，从本书衍生的材料必须具有相同的许可证（或与之兼容的许可证）。有关此许可证的更多信息，请参见 <https://creativecommons.org/licenses/by-sa/4.0/>。

动机

重新执行问题 计算机科学的一个基本目标是**计算完整性**：

一方如何验证另一方正确地执行了一个给定的计算？

实现这一目标的一种自然方法是**重新执行计算**，并检查这得到的结果是否与声称的结果相同。

例如，考虑一个计算任务：计算 1 亿以内的质数个数。Alice 通过质数计数算法（例如埃拉托色尼筛法）确定答案是 50,847,534，并发布计算声明“1 亿以内的质数个数是 50,847,534”。Bob 如何检查 Alice 的计算声明是否正确？一个选择是 Bob 重新运行 Alice 使用的算法（或其他质数计数算法），并检查 1 亿以内的质数是否确实是 50,847,534。

然而，在许多情况下，重新执行**不是一个选项**。

- 一方可能无法承担（或没有足够的资源）重新执行与计算声明相关的计算。例如，使用最先进的质数计数算法在 2022 年计算 10^{29} 以内的质数个数要花费 90 多核年和超过 1TB 的峰值内存使用；¹ 这样的计算不易重新执行。
- 此外，计算可能涉及无法与他人共享的私有输入，因此无法重新执行计算。以下是一个涉及 RSA 密钥对的例子。回忆一下，RSA 密钥对包括一个包含两个质数（从合适的分布中抽样）的私钥和一个包含它们乘积的公钥；这样的密钥对用于加密方案和基于 RSA 假设的数字签名方案。考虑声明“我知道两个质数，它们的乘积是 RSA 公钥 N ”。揭示这两个质数可以让其他人轻松验证该声明。然而，这样做会泄露与公钥对应的私匙。

于是我们面临重新执行问题：

如何在不重新执行的情况下确保计算完整性？

乍一看这似乎是不可能的。计算复杂性中的分层定理指出，一般来说，没有办法减少确定性计算输出所使用的时间或空间。² 在不重新执行的情况下确保计算完整性似乎是一个难以实现的目标。

避免重新执行 令人惊讶的是，通过要求提出计算声明的一方同时提供该声明的计算完整性的密码学证明，就可以避免重新执行。（特别地，这规避了前面提到的限制，以及在此尚未提及的其他限制。）非正式地，一方可以通过证明者算法生成密码学证明来认证计算的正确性。随后，任何人都可以通过验证算法检查密码学证明，而非重新执行原始计算。安全性适用于资源适当受限的敌手（稍后会详细介绍）。

但是，密码学证明的验证如何“优于”重新执行呢？这是由于密码学证明可以实现的优良性质。

- **简洁性。**验证算法可以比重重新执行原始计算快得多。这种特性，称为简洁性，使得计算能力弱的一方可以验证计算复杂的计算的正确性。
- **零知识。**密码学证明不会泄露任何计算中使用的私有输入的信息。这种特性称为零知识，使得其他人可以验证涉及秘密的计算的正确性。（这些秘密只有执行计算并生成密码学证明的一方知道。）

满足（某种形式的）简洁性的密码学证明通常称为简洁证明，如果是非交互式的，则称为 SNARG。一个流行的概念是 zkSNARK，即零知识简洁非交互知识论证。这些概念将在本书中引入并定义。

应用 密码学证明通常用于学术研究，设计密码学原语、协议和系统。更令人兴奋的是，密码学证明已经在学术界之外找到了应用：它们在许多无法重新执行的场景中得到了广泛的实际应用。截至写作时，它们是应用密码学中最热门的话题之一。

例如，密码学证明在设计安全分布式系统（如区块链）中起着关键作用。它们被用于实现可扩展性和隐私的目标，简要介绍如下。

- **可扩展性。**简洁性性质通过将昂贵的计算移出网络节点，从而增加对等网络中的交易吞吐量。智能合约系统（例如，以太坊）中的计算既慢又昂贵：每个计算步骤都由网络中的每个节点重新执行，因此网络受限于最慢的节点，用户为每个计算步骤支付高昂的价格。一类称为“基于证明的 rollups”的架构将计算移出链外，用户将其计算请求发送给聚合者，聚合者定期生成关于用户计算批次的密码学证明；然后智能合约系统验证密码学证明并授权一个

¹ 参见 <https://oeis.org/A006880> 和 <https://www.mersenneforum.org/showthread.php?t=20473>

² 例如，确定性多带图灵机分层定理指出，对于每个时间可构造函数 $t(n)$ ，复杂性类 $\text{DTIME}(t(n))$ 严格大于复杂性类 $\text{DTIME}(o(\frac{t(n)}{\log t(n)}))$ 。换句话说，有些计算任务的输出可以在时间 $t(n)$ 内确定，但不能在时间 $o(\frac{t(n)}{\log t(n)})$ 内确定。其他资源和计算模型也存在其他分层定理。

反映批次中所有计算的状态转换。简洁性确保检查密码学证明比检查它所证明的计算便宜得多。因此，让网络中的每个节点仅检查密码学证明要便宜得多，并且使系统能够扩展到更多的用户和更大的计算。

- **隐私。**零知识性质允许设计具有强用户隐私的对等网络。非正式地，零知识性质使得网络中的一方能够广播关于涉及其私有输入的计算的密码学证明。例如，计算可能授权从一方向另一方转移数字资产，但不透露这些方的身份或数字资产的类型和数量。这种隐私保护功能对密码货币和其他去中心化金融应用至关重要，因为交易通常涉及高度敏感的信息，不应向整个网络公开。

这些及其他应用在学术界、工业界和政府的多个社区中引起了对密码学证明的极大兴趣。这种更广泛的兴趣需要关于密码学证明的材料更易于访问和系统化，以有效地满足超出少数专家的受众的需求。

1 数学背景

我们介绍本书中使用的基本数学记号和事实。

1.1 符号约定

集合 我们用 $|S|$ 表示集合 S 的基数（即元素个数）；如果 S 的元素个数为无限，则 $|S| := \infty$ 。我们用 $\mathbb{R}$ 表示实数集，用 $\mathbb{N}$ 表示正整数集 $1, 2, \dots$ ，并用 $[n]$ 表示正整数集 $1, 2, \dots, n$ 。对于 $k \in 0, 1, \dots, n$ ，我们用 $\binom{[n]}{k}$ 表示所有从 $[n]$ 中选取 k 个元素的子集所组成的集合；这类子集的数量为 $\binom{n}{k} := \frac{n!}{k!(n-k)!}$ 。我们用 $x_1, \dots, x_n$ 表示包含元素 $x_1, \dots, x_n$ 的集合；该集合的基数为 $x_1, \dots, x_n$ 中不同元素的个数（重复元素只计一次）。给定一个集合 S ，我们记 $x_{i \in S}$ 表示这样一个集合：对每个 $i \in S$ ，都包含元素 x_i ；例如， $x_{i \in [n]} = x_1, \dots, x_n$ 。

元组 元组是元素的有序列表。我们用 $(x_i)_{i \in [n]}$ 表示元组 $(x_1, \dots, x_n)$ ；更一般地， $(x_i)_{i \in S}$ 表示一个元组，其条目根据给定集合 S 的惯例顺序进行索引。元组中重复的元素会在相应位置多次出现。通过嵌套括号可以得到元组的元组；例如， $((x_i, y_i, z_i))_{i \in [n]}$ 表示元组的元组 $((x_1, y_1, z_1), \dots, (x_n, y_n, z_n))$ 。如果 x 是一个条目由集合 S 索引的元组，则对每个 $i \in S$ ， $x[i]$ 表示 x 的第 i 个条目；更一般地，对于子集 $T \subseteq S$ ， $x[T]$ 表示将 x 中索引在 T 内的条目取出后构成的元组。例如，如果 $x = (x_i)_{i \in 1,4,7}$ ，则 $x[7] = x_7$ ， $x[1, 7] = (x_i)_{i \in 1,7}$ 。

字符串 设 Σ 为一个有限字母表（即有限集合）。在 Σ 上的字符串是一个定义在某个有序集合 D 上的函数 $x: D \rightarrow \Sigma$ ，其取值属于 Σ ；特别地， x 是一个元组 $(x_i)_{i \in D}$ ，其中每个条目 x_i 都是字母表 Σ 的元素。通常 $D = [n]$ ($n \in \mathbb{N}$)，此时我们可以通过列举符号的序列来书写字符串；例如，001 表示字符串 $x: [3] \rightarrow \{0, 1\}$ ，其中 $x_1 = 0$ 、 $x_2 = 0$ 、 $x_3 = 1$ 。我们用 $\parallel$ 表示字符串连接；例如 $00\parallel 1$ 表示字符串 001。

对于每个 $n \in \mathbb{N}$ ，我们用 Σ^n 表示所有长度为 n 的 Σ 上的字符串的集合：

$$\Sigma^n := \{x: [n] \rightarrow \Sigma\} = \{(x_i)_{i \in [n]} : \forall i \in [n], x_i \in \Sigma\}.$$

我们用 $\Sigma^* := \cup_{n \in \mathbb{N}} \Sigma^n$ 表示所有有限长度 Σ 上的字符串的集合。

给定一个有序集合 D ，我们用 Σ^D 表示所有长度为 $|D|$ 且条目由 D 的元素索引的 Σ 上的字符串的集合：

$$\Sigma^D := \{x: D \rightarrow \Sigma\} = \{(x_i)_{i \in D} : \forall i \in D, x_i \in \Sigma\}.$$

例如，如果 $\Sigma = \{0, 1\}$ 且 $D = 1, 8, 9$ ，则字符串 $x \in \Sigma^D = \{0, 1\}^{1,8,9}$ 可以具有以下取值： $x[1] = 0$ 、 $x[8] = 1$ 、 $x[9] = 0$ 。

位长 二进制字符串是字母表 $\Sigma = \{0, 1\}$ 上的字符串。我们用 $\text{len}(x)$ 表示二进制字符串 $x \in \{0, 1\}^*$ 的长度；例如 $\text{len}(001) = 3$ 。如果 x 不是二进制字符串，则 $\text{len}(x)$ 表示将 x 适当表示为二进制字符串后的长度。例如，若 $x \in \Sigma^n$ ，则可通过将 x 中的每个符号表示为一个 $\lceil \log_2 |\Sigma| \rceil$ 位的二进制字符串，得到 $\text{len}(x) = n \cdot \lceil \log_2 |\Sigma| \rceil$ 。若 $x \in \Sigma^D$ ，则将 x 视为由 $n = |D|$ 对 (i, x_i) 组成的列表，其中每对由一个 $\lceil \log_2 |D| \rceil$ 位的二进制字符串（编码索引 i ）和一个 $\lceil \log_2 |\Sigma| \rceil$ 位的二进制字符串（编码符号 x_i ）组成，从而 $\text{len}(x) = n \cdot (\lceil \log_2 |D| \rceil + \lceil \log_2 |\Sigma| \rceil)$ 。

抽样 我们用 $a \leftarrow D$ 表示按照分布 D 抽样得到 a 的过程。类似地，给定一个有限集合 S ，我们用 $a \leftarrow S$ 表示按照对 S 上的均匀分布抽样得到 a 的过程。

渐近符号 我们使用标准的渐近符号。例如，给定两个函数 $f, g: \mathbb{N} \rightarrow \mathbb{R}$ ，我们分别用 $f(n) = O(g(n))$ 和 $f(n) = \Omega(g(n))$ 表示存在常数 $c > 0$ ，使得对所有足够大的 n ，都有 $f(n) \leq c \cdot g(n)$ 和 $f(n) \geq c \cdot g(n)$ 。此外，我们用 $f(n) = o(g(n))$ 和 $f(n) = \omega(g(n))$ 分别表示对每个常数 $c > 0$ ，当 n 足够大时，均有 $f(n) \leq c \cdot g(n)$ 和 $f(n) \geq c \cdot g(n)$ 。我们还使用符号 $\text{poly}(n)$ 表示某个未指明的多项式上界函数，即 $n^{O(1)}$ ；更一般地， $\text{poly}(n_1, n_2, \dots)$ 表示 $(n_1 + n_2 + \dots)^{O(1)}$ 。

语言与关系 关系 $\mathcal{R}$ 是由实例-证据对 (x, w) 所组成的集合。对应的语言 $\mathcal{L}(\mathcal{R})$ 是那些存在证据 w 使得 $(x, w) \in \mathcal{R}$ 的实例 x 的集合。我们用 $|x|$ 表示实例的大小；这可以是描述 x 所需的比特数（即 $\text{len}(x)$ ）或其他适用于关系 $\mathcal{R}$ 的度量。

算法 我们非正式地将算法称为在适当计算模型上运行的过程。如果我们不关心算法的时间或空间复杂度（例如当考虑无界对手时），则底层计算模型无关紧要。（无论是单带图灵机、多带图灵机、寄存器机或其他均可。）如果我们讨论算法的时间或空间复杂度，则假设算法在类似现代计算机的寄存器机上运行（具有有限指令集并能对具有适当字长的内存进行读写随机访问）。我们同时考虑确定性算法和（访问一定长度随机位串的）随机化算法。

如果一个算法的运行时间由其输入大小的某个多项式上界，则称该算法在多项式时间内运行。（若算法有多个输入，则考虑它们大小之和。）

交互式算法 如果算法 A 在多次输入-输出调用之间保持状态，则称 A 为交互式。对于输入序列 $a_1, a_2, a_3, \dots$ ， A 的输出如下：

$$b_1 \leftarrow A(a_1), ; b_2 \leftarrow A(a_2), ; b_3 \leftarrow A(a_3), ; \dots$$

有时，将一次执行传递到下一次执行的状态显式化会更有帮助，此时我们将算法 A 视为无状态：

$$(b_1, \text{aux}_1) \leftarrow A(a_1), ; (b_2, \text{aux}_2) \leftarrow A(\text{aux}_1, a_2), ; (b_3, \text{aux}_3) \leftarrow A(\text{aux}_2, a_3), ; \dots$$

黑盒 有时我们描述一种算法，它将其他算法作为输入，并且仅依赖于它们的输入-输出功能（通过在特定输入上运行并获得相应输出），而不使用它们的具体实现。我们称此时这些算法被用作黑盒。

设 A 和 B 为算法。若 A 在某些输入上多次调用 B ，并且除此之外不“查看” B 的实现细节，则称 A 将 B 用作黑盒，记为 $A(\mathcal{O}_B)$ 。更正式地说，若对于任意表示相同功能的两个算法 B_1 和 B_2 ，都有 $A(\mathcal{O}_{B_1}) = A(\mathcal{O}_{B_2})$ ，则满足上述定义。

若 B 是交互式算法，则定义 $\mathcal{O}_B$ 为 B 的无状态表示的黑盒，即给定 B 的内部状态和输入，返回相应输出及新的内部状态的函数的黑盒。具体地， $A(\mathcal{O}_B)$ 可以在输入序列 $a_1, a_2, a_3, \dots$ 上与 B 交互，如下所示：

$$(b_1, \text{aux}_1) \leftarrow \mathcal{O}_B(a_1), ; (b_2, \text{aux}_2) \leftarrow \mathcal{O}_B(\text{aux}_1, a_2), ; (b_3, \text{aux}_3) \leftarrow \mathcal{O}_B(\text{aux}_2, a_3), ; \dots$$

在上述黑盒调用序列中存在一个微妙的技术细节。从 $(B$ 的无状态表示中) 传递给 A 的内部状态 $(\text{aux}_i)_i$ 可能包含关于算法 B 的非黑盒信息（例如 B 的代码）；此外，这些内部状态可能很大（例如，如果 B 效率低下，则可能呈指数规模），这会影响 A 的效率。然而， A 只对 B 的输入-输出行为感兴趣，我们希望确

一般情况称为容斥原理，它提供了一种将 n 个事件 $E_1, \dots, E_n$ 的并集的概率表示为它们所有交集概率（带符号）之和的方法。

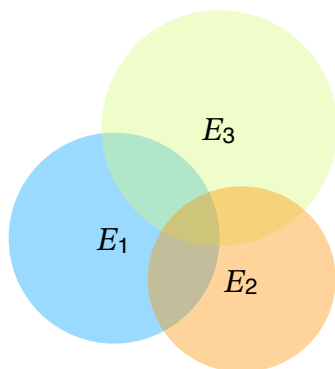


Figure 1.1: 事件 E_1, E_2, E_3 的两两和三重交集示意图。

引理 1.1.1 (inclusion – exclusion principle). 对于任意 n 个事件 $E_1, \dots, E_n$, 有

$$\Pr[\vee_{i \in [n]} E_i] = \sum_{k \in [n]} \left((-1)^{k+1} \cdot \sum_{\substack{I \subseteq [n] \\ \text{with } |I|=k}} \Pr[\wedge_{i \in I} E_i] \right).$$

上述引理中的等式可能不便于使用, 因为它涉及了大量项。下面关于 $\vee_{i \in [n]} E_i$ 的概率的陈述更易使用: 它分别在事件 E_i 的个体概率和它们两两交集 $E_i \wedge E_j$ 的概率基础上给出了一个下界和上界。

引理 1.1.2 (simple inclusion – exclusion principle). 对于任意 n 个事件 $E_1, \dots, E_n$,

$$\sum_{i \in [n]} \Pr[E_i] - \sum_{\substack{i, j \in [n] \\ \text{with } i \neq j}} \Pr[E_i \wedge E_j] \leq \Pr[\vee_{i \in [n]} E_i] \leq \sum_{i \in [n]} \Pr[E_i].$$

Proof. 我们对事件数量 n 进行归纳证明。

基例: $n \in \{1, 2, 3\}$ 。当 $n = 1$ 时, 由于引理中的各表达式均为 $\Pr[E_1]$, 结论显然成立。至于 $n = 2$ 和 $n = 3$ 的情况, 我们在引理之前的讨论中已分别加以证明。

归纳步: $n > 1$ 。假设对于任意 $n' \in [n - 1]$, 该引理都已成立。定义事件

$$E^* := \vee_{i \in [n-1]} E_i.$$

由对 $n' = n - 1$ 的归纳假设, 可得

$$\sum_{i \in [n-1]} \Pr[E_i] - \sum_{\substack{i, j \in [n-1] \\ \text{with } i \neq j}} \Pr[E_i \wedge E_j] \leq \Pr[E^*] \leq \sum_{i \in [n-1]} \Pr[E_i]. \quad (1.1)$$

首先证明上界。对事件 E^* 和 E_n 应用 $n' = 2$ 的归纳假设, 并结合 Equation 1.1, 得到

$$\Pr[E^* \vee E_n] \leq \Pr[E^*] + \Pr[E_n] \leq \sum_{i \in [n-1]} \Pr[E_i] + \Pr[E_n] = \sum_{i \in [n]} \Pr[E_i].$$

接下来证明下界。将已证上界应用于事件集合 $\{E_i \wedge E_n\}_{i \in [n]}$, 得到

$$\Pr\left[\bigvee_{i \in [n]} (E_i \wedge E_n)\right] \leq \sum_{i \in [n]} \Pr[E_i \wedge E_n]. \quad (1.2)$$

再对 E^* 和 E_n 应用 $n' = 2$ 的归纳假设, 结合 Equations 1.1 and 1.2, 可得

$$\begin{aligned} & \Pr[E^* \vee E_n] \\ & \geq \Pr[E^*] + \Pr[E_n] - \Pr[E^* \wedge E_n] \\ & \geq \left(\sum_{i \in [n-1]} \Pr[E_i] - \sum_{\substack{i, j \in [n-1] \\ \text{with } i \neq j}} \Pr[E_i \wedge E_j] \right) + \Pr[E_n] - \Pr[E^* \wedge E_n] \\ & = \sum_{i \in [n]} \Pr[E_i] - \sum_{\substack{i, j \in [n-1] \\ \text{with } i \neq j}} \Pr[E_i \wedge E_j] - \Pr\left[\bigvee_{i \in [n]} (E_i \wedge E_n)\right] \\ & \geq \sum_{i \in [n]} \Pr[E_i] - \sum_{\substack{i, j \in [n] \\ \text{with } i \neq j}} \Pr[E_i \wedge E_j]. \end{aligned}$$

□

我们通过涉及随机变量的表达式来描述事件。在本书中, 随机变量取值为二进制字符串或实数。在后者情况下, 二元随机变量指仅取 0 或 1 值的随机变量。

Random variables 如果 X 是一个取值于有限集合 $S \subseteq \mathbb{R}$ 的随机变量, 则其期望定义为:

$$\mathbb{E}[X] := \sum_{x \in S} x \cdot \Pr[X = x].$$

随机变量 X 和 Y 独立当且仅当对任意 x, y 有

$$\Pr[X = x \wedge Y = y] = \Pr[X = x] \cdot \Pr[Y = y].$$

更一般地, 随机变量 $X_1, \dots, X_n$ 独立当且仅当对任意 $x_1, \dots, x_n$ 有

$$\Pr\left[\bigwedge_{i \in [n]} X_i = x_i\right] = \prod_{i \in [n]} \Pr[X_i = x_i].$$

大数定律指出, 给定一系列相互独立且同分布的随机变量 $\{X_i\}_{i \in \mathbb{N}}$, $\mathbb{E}[X]$ 是 $\frac{1}{n} \sum_{i \in [n]} X_i$ 在 $n \rightarrow \infty$ 时的极限。 $\frac{1}{n} \sum_{i \in [n]} X_i$ 与 $\mathbb{E}[X]$ 的偏离随 n 变化的情况由 Chernoff 不等式刻画。其中一个例子是以下(加性) Chernoff 界。

引理 1.1.3 (Chernoff bound). 令独立的二元随机变量 $X_1, \dots, X_n$ 满足 $\Pr[X_i = 1] = p_i$ 。令 $X := \sum_{i \in [n]} X_i$ 和 $\mu := \mathbb{E}[X] = \sum_{i \in [n]} p_i$ 。对于任意 $\delta \in (0, 1)$, 有

$$\Pr[|X - \mu| \geq \delta \cdot \mu] \leq 2 \cdot e^{-\frac{\mu \delta^2}{3}}.$$

Statistical distance 我们以全变差距离来度量两个随机变量的统计距离，如下定义。

定义 1.1.4. 取值于有限集合 S 的随机变量 X 和 Y 之间的统计距离定义为

$$\Delta(X, Y) := \frac{1}{2} \sum_{a \in S} |\Pr[X = a] - \Pr[Y = a]|. \quad (1.3)$$

等价地，统计距离也可定义为

$$\Delta(X, Y) := \max_{S' \subseteq S} |\Pr[X \in S'] - \Pr[Y \in S']|. \quad (1.4)$$

我们将证明上述两种定义 (Equations 1.3 and 1.4) 是等价的。

论断 1.1.5. 使用 Equation 1.3 给出的定义与使用 Equation 1.4 给出的定义等价。

Proof. 定义子集

$$S_0 := \{a \in S : \Pr[X = a] \geq \Pr[Y = a]\},$$

并令其补集为 $S_1 := S \setminus S_0$ 。注意

$$\Pr[X \in S_0] + \Pr[X \in S_1] = 1, \quad \Pr[Y \in S_0] + \Pr[Y \in S_1] = 1.$$

因此

$$\Pr[X \in S_0] - \Pr[Y \in S_0] = \Pr[Y \in S_1] - \Pr[X \in S_1]. \quad (1.5)$$

可以证明 S_0 或 S_1 最大化 $\max_{S' \subseteq S} |\Pr[X \in S'] - \Pr[Y \in S']|$ 。然后，利用 Equation 1.5，得到

$$\Pr[Y \in S_1] - \Pr[X \in S_1] = \max_{S' \subseteq S} |\Pr[X \in S'] - \Pr[Y \in S']| = \Pr[X \in S_0] - \Pr[Y \in S_0].$$

最后，我们有

$$\begin{aligned} \Delta(X, Y) &:= \frac{1}{2} \sum_{a \in S} |\Pr[X = a] - \Pr[Y = a]| \\ &= \frac{1}{2} \left(\sum_{a \in S_0} (\Pr[X = a] - \Pr[Y = a]) + \sum_{a \in S_1} (\Pr[Y = a] - \Pr[X = a]) \right) \\ &= \frac{1}{2} (\Pr[X \in S_0] - \Pr[Y \in S_0] + \Pr[Y \in S_1] - \Pr[X \in S_1]) \\ &= \Pr[X \in S_0] - \Pr[Y \in S_0] \\ &= \max_{S' \subseteq S} |\Pr[X \in S'] - \Pr[Y \in S']|. \end{aligned}$$

□

论断 1.1.6. 统计距离函数具有以下性质：

1. 对任意随机变量 X , $\Delta(X, X) = 0$ 。
2. 对任意随机变量 X, Y , $0 \leq \Delta(X, Y) \leq 1$ 。
3. 对任意随机变量 X, Y, Z ,

$$\Delta(X, Z) \leq \Delta(X, Y) + \Delta(Y, Z).$$

Proof. 前两条性质由定义即得。我们证明第三条如下:

$$\begin{aligned}
 \Delta(X, Z) &= \frac{1}{2} \sum_a |\Pr[X = a] - \Pr[Z = a]| \\
 &= \frac{1}{2} \sum_a |(\Pr[X = a] - \Pr[Y = a]) + (\Pr[Y = a] - \Pr[Z = a])| \\
 &\leq \frac{1}{2} \sum_a |\Pr[X = a] - \Pr[Y = a]| + \frac{1}{2} \sum_a |\Pr[Y = a] - \Pr[Z = a]| \\
 &= \Delta(X, Y) + \Delta(Y, Z).
 \end{aligned}$$

□

论断 1.1.7. 对于任意随机变量 X, Y 和定义在取值域 S 上的函数 f , 有

$$\Delta(f(X), f(Y)) \leq \Delta(X, Y).$$

Proof.

$$\begin{aligned}
 \Delta(f(X), f(Y)) &= \frac{1}{2} \sum_{b \in f(S)} |\Pr[f(X) = b] - \Pr[f(Y) = b]| \\
 &= \frac{1}{2} \sum_{b \in f(S)} \left| \sum_{a \in f^{-1}(b)} (\Pr[X = a] - \Pr[Y = a]) \right| \\
 &\leq \frac{1}{2} \sum_{b \in f(S)} \sum_{a \in f^{-1}(b)} |\Pr[X = a] - \Pr[Y = a]| \\
 &\leq \frac{1}{2} \sum_{a \in S} |\Pr[X = a] - \Pr[Y = a]| \\
 &\leq \Delta(X, Y).
 \end{aligned}$$

□

假设我们有一个随机变量 X 和两个依赖于 X 的随机变量家族 $\{Y(x)\}_x$ 和 $\{Y'(x)\}_x$ 。定义随机变量 Y 为先从 X 取样 x , 再从 $Y(x)$ 中取样; Y' 类似。若

$$\Pr_{x \leftarrow X} [\Delta(Y(x), Y'(x)) \geq \delta] \leq \epsilon,$$

则

$$\Delta((X, Y), (X, Y')) \leq \delta + \epsilon.$$

论断 1.1.8. 如上所述, 若 $\Pr_{x \leftarrow X} [\Delta(Y(x), Y'(x)) \geq \delta] \leq \epsilon$, 则

$$\Delta((X, Y), (X, Y')) \leq \delta + \epsilon.$$

Proof. 定义

$$S := \{x : \Delta(Y(x), Y'(x)) \geq \delta\}.$$

则

$$\begin{aligned}
\Delta((X, Y), (X, Y')) &= \frac{1}{2} \sum_{x, y} |\Pr[(X, Y) = (x, y)] - \Pr[(X, Y') = (x, y)]| \\
&= \frac{1}{2} \sum_{x, y} |\Pr[X = x](\Pr[Y(x) = y] - \Pr[Y'(x) = y])| \\
&= \sum_x \Pr[X = x] \cdot \Delta(Y(x), Y'(x)) \\
&= \sum_{x \in S} \Pr[X = x] \cdot \Delta(Y(x), Y'(x)) + \sum_{x \notin S} \Pr[X = x] \cdot \Delta(Y(x), Y'(x)) \\
&\leq \sum_{x \in S} \Pr[X = x] \cdot 1 + \sum_{x \notin S} \Pr[X = x] \cdot \delta \\
&\leq \epsilon + \delta.
\end{aligned}$$

□

论断 1.1.9. 令 X 和 Y 为同一概率空间上的随机变量, E_1 和 E_2 为两事件且 $\Pr[E_1] = \Pr[E_2]$ 。则

$$\Delta(X, Y) \leq \Delta((X | E_1), (Y | E_2)) + \Pr[\overline{E_1}].$$

Proof. 设 S 为这两个随机变量的取值域。对任意子集 $S' \subseteq S$, 由全概率公式有:

$$\begin{aligned}
&|\Pr[X \in S'] - \Pr[Y \in S']| \\
&= \left| \Pr[X \in S' | E_1] \cdot \Pr[E_1] + \Pr[X \in S' | \overline{E_1}] \cdot \Pr[\overline{E_1}] \right. \\
&\quad \left. - \Pr[Y \in S' | E_2] \cdot \Pr[E_2] - \Pr[Y \in S' | \overline{E_2}] \cdot \Pr[\overline{E_2}] \right| \\
&= \left| \Pr[E_1](\Pr[X \in S' | E_1] - \Pr[Y \in S' | E_2]) \right. \\
&\quad \left. + \Pr[\overline{E_1}](\Pr[X \in S' | \overline{E_1}] - \Pr[Y \in S' | \overline{E_2}]) \right| \\
&\leq \Pr[E_1] |\Pr[X \in S' | E_1] - \Pr[Y \in S' | E_2]| \\
&\quad + \Pr[\overline{E_1}] |\Pr[X \in S' | \overline{E_1}] - \Pr[Y \in S' | \overline{E_2}]| \\
&\leq |\Pr[X \in S' | E_1] - \Pr[Y \in S' | E_2]| + \Pr[\overline{E_1}].
\end{aligned}$$

因此, 由等价定义 Equation 1.4, 得

$$\begin{aligned}
\Delta(X, Y) &:= \max_{S' \subseteq S} |\Pr[X \in S'] - \Pr[Y \in S']| \\
&\leq \max_{S' \subseteq S} |\Pr[X \in S' | E_1] - \Pr[Y \in S' | E_2]| + \Pr[\overline{E_1}] \\
&= \Delta((X | E_1), (Y | E_2)) + \Pr[\overline{E_1}].
\end{aligned}$$

□

1.2 概率实验

我们引入用于定义这些实验上随机变量和事件的记号。

- 我们记 $\{x \mid x \leftarrow A\}$ 表示根据随机算法 A 采样得到字符串 x 的随机变量。
- 我们记 $\Pr[f(x) = 1 \mid x \leftarrow A]$ 表示当 x 根据随机算法 A 采样时, 对于给定的布尔函数 f , 事件 $f(x) = 1$ 的概率。
- 我们记 $\Pr[D(x) = 1 \mid x \leftarrow A]$ 表示当 x 根据随机算法 A 采样, 并且 D 是随机算法时, 事件 $D(x) = 1$ 的概率。在此情况下, 概率在 A 和 D 的随机性上取。

有时我们在表达式内部隐式地定义算法 A 或布尔函数, 使用伪代码和逻辑表达式。例如, 我们可以写

$$\{(y, z) \mid \begin{array}{l} y \leftarrow B \\ z \leftarrow C(y) \end{array}\}$$

表示随机变量 $x := (y, z)$, 由算法 A 实现: 先根据 B 采样得到 y , 再以 y 为输入根据 C 采样得到 z , 并输出对 (y, z) 。我们也可以写

$$\Pr[y = z \mid \begin{array}{l} y \leftarrow B \\ z \leftarrow C(y) \end{array}]$$

表示如上采样得到的随机变量 $x = (y, z)$ 满足布尔函数 $f(x) = 1$ (当且仅当 $y = z$) 的概率。

本书中概率实验的主要目的是通过指定对手的行为并给出其输出的“获胜条件”来描述密码方案的安全性属性。

1.3 辅助等式 (不等式)

我们陈述本书中将反复使用的若干等式和不等式。

引理 1.3.1 (二项式系数的界). 对于任意 $n, k \in \mathbb{N}$ 且 $1 \leq k \leq n$,

$$\left(\frac{n}{k}\right)^k \leq \binom{n}{k} \leq \left(\frac{e \cdot n}{k}\right)^k.$$

引理 1.3.2 (等差级数). 对于任意 $a, d \in \mathbb{R}$,

$$\sum_{i=1}^n (a + (i-1)d) = na + \frac{n(n-1)}{2}d.$$

引理 1.3.3 (等比级数). 对于任意 $r \in (0, 1)$ 和 $a \in \mathbb{R}$,

$$\sum_{i=1}^n a r^{i-1} = a \frac{r^n - 1}{r - 1}.$$

引理 1.3.4. 令 $\delta_1, \dots, \delta_n \in [0, 1]$ 且满足 $\sum_{i=1}^n \delta_i = 1$ 。则

$$\sum_{i=2}^n \delta_{i-1} \cdot \delta_i \leq 1/4.$$

Proof. 我们首先注意到要给出 1 的上界相对容易。由于每个 $\delta_i \leq 1$, 可得

$$\sum_{i=2}^n \delta_{i-1} \cdot \delta_i \leq \sum_{i=2}^n \delta_i \leq 1.$$

下面的证明致力于给出更紧的 $1/4$ 上界。

第一步, 我们证明当除三个变量外其余均为零时, 表达式取得最大值。令索引 $i \in [n]$ 是使 $\delta_{i-1} + \delta_{i+1}$ 最大的最小下标。对于任意 $j \in [n] \setminus \{i-1, i, i+1\}$ 且 $\delta_j > 0$, 我们将 δ_j 的全部权重移动到 δ_i 并展示该操作不会减小表达式。定义新的序列

$$\delta'_k := \begin{cases} 0, & \text{若 } k = j; \\ \delta_i + \delta_j, & \text{若 } k = i; \\ \delta_k, & \text{其他情况.} \end{cases}$$

然后, 原式与新序列下的式子之差仅在含 i 或 j 的项上。我们需验证

$$\delta_{j-1}\delta_j + \delta_j\delta_{j+1} + \delta_{i-1}\delta_i + \delta_i\delta_{i+1} \leq \delta'_{j-1}\delta'_j + \delta'_j\delta'_{j+1} + \delta'_{i-1}\delta'_i + \delta'_i\delta'_{i+1}.$$

注意 $\delta'_j = 0$ 且 $\delta'_i = \delta_i + \delta_j$, 故不等式化简为

$$\delta_{j-1} + \delta_{j+1} \leq \delta_{i-1} + \delta_{i+1},$$

这正是 i 的选择所保证的。因此, 除 $\{i-1, i, i+1\}$ 外, 其余 $\delta_k = 0$ 。

在此基础上, 原式化简为

$$\delta_{i-1}\delta_i + \delta_i\delta_{i+1} = \delta_i(\delta_{i-1} + \delta_{i+1}) = \delta_i(1 - \delta_i) \leq \frac{1}{4},$$

完成证明。 □